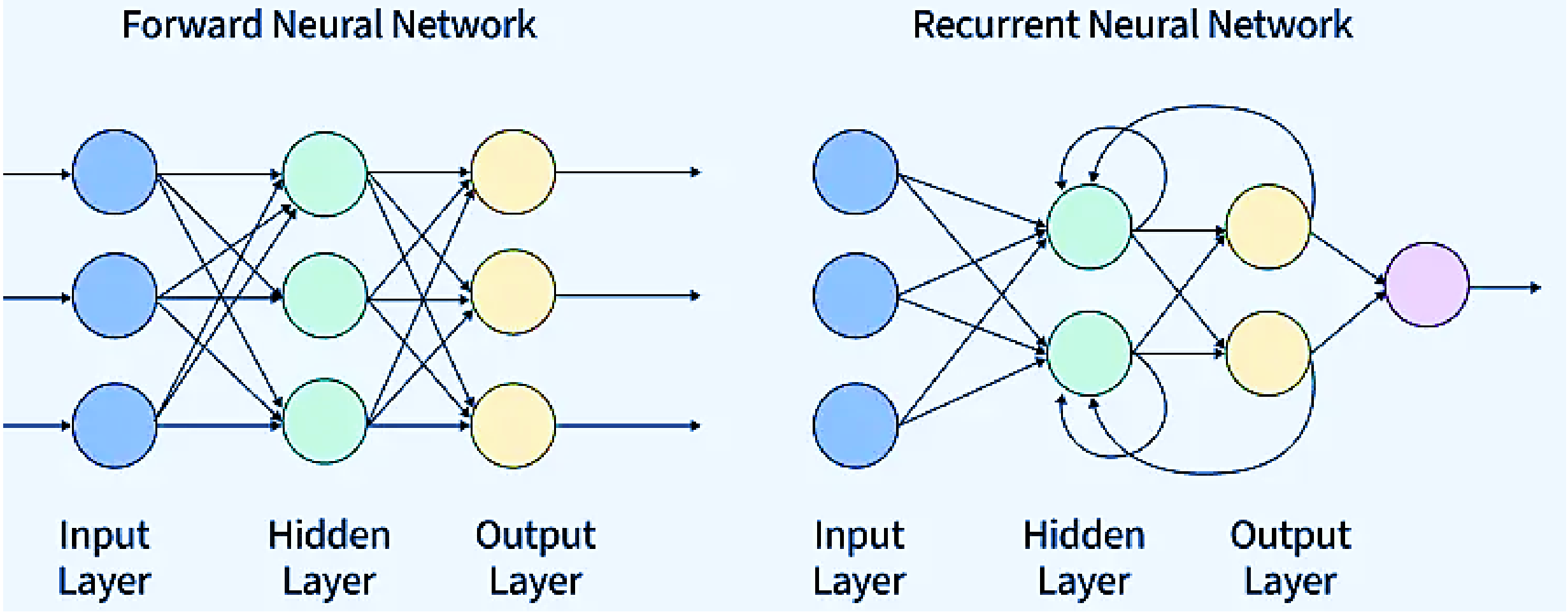
Deep Learning (23AID211)

Recurrent Neural Network (RNN)
&
Long-Short- Term-Memory (LSTM)

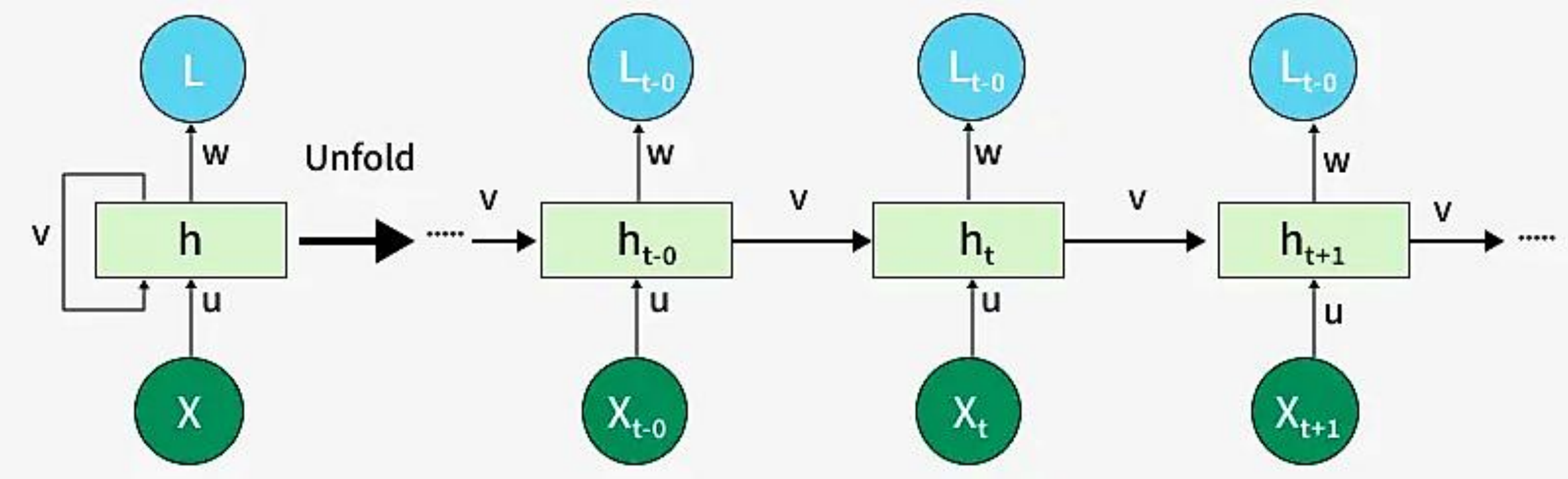
By
Dr. Vivek Patel
(B.Tech, M.Tech, Ph.D)
School of Artificial Intelligence, Amrita Vishwa Vidyapeetham
Faridabad Campus, Delhi NCR, India

Recurrent Neural Network (RNN)



Recurrent Neural Network (RNN)

Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by retaining information from previous steps.



Recurrent Neuron

In RNNs the hidden state Hi is calculated for every input Xi to retain sequential dependencies. The computations follow these core formulas:

1. Hidden State Calculation:

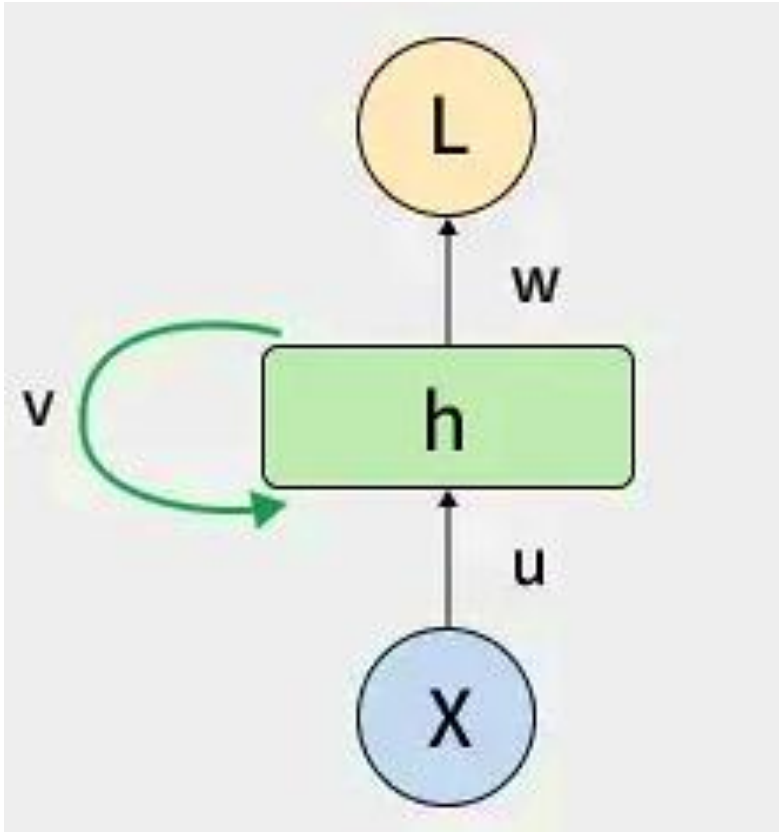
$$h = \sigma(U \cdot X + W \cdot h_{t-1} + B)$$

Here:

- h represents the current hidden state.
- U and W are weight matrices.
- B is the bias.

2. Output Calculation:

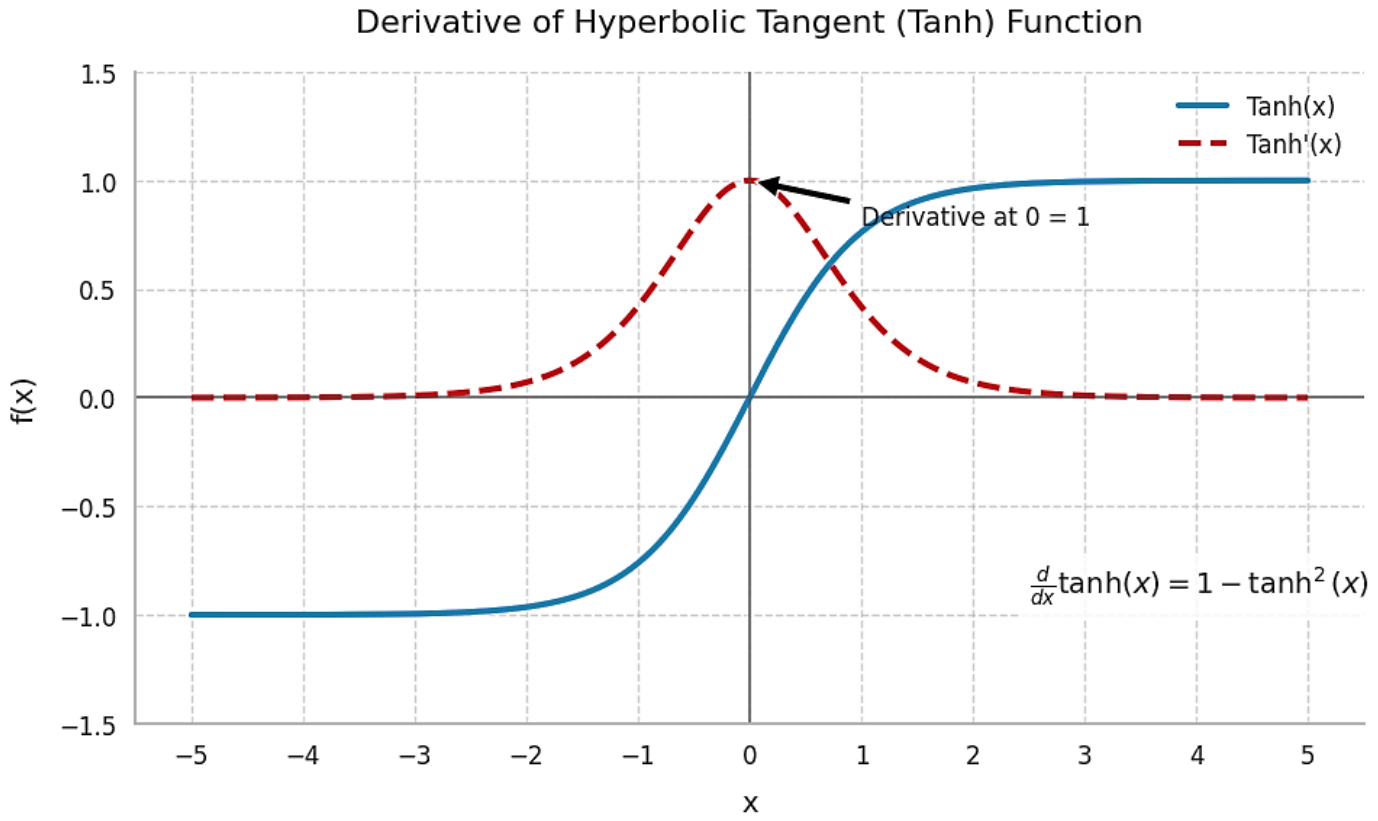
$$Y = O(V \cdot h + C)$$



Tanh (Hyperbolic Tangent) Activation Function

The **Tanh (Hyperbolic Tangent)** activation function is fundamental to Recurrent Neural Networks (RNNs) due to its ability to manage the internal state and gradient flow during sequential processing.

$$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x)$$



Why Use Tanh in Neural Networks?

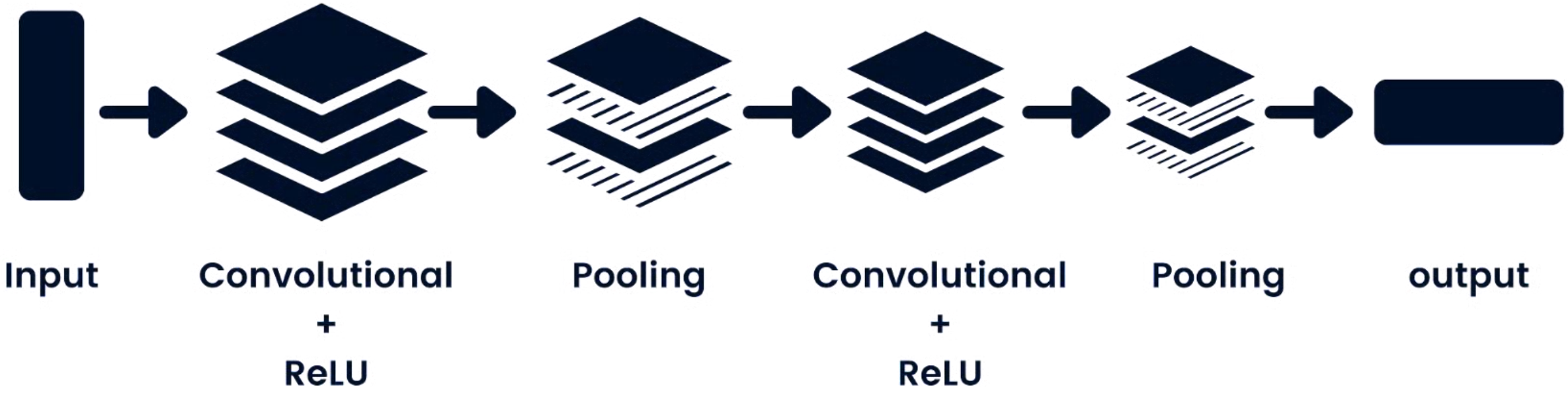
Non-linearity: Tanh introduces non-linearity to the model, which allows neural networks to learn complex patterns and relationships in the data.

Centered Around Zero: The output of the tanh function is centered around 0, unlike the sigmoid function. This tanh activation function is more useful because the mean of the output is closer to zero, leading to more efficient training and faster convergence.

Gradient Behavior: Tanh helps to mitigate the vanishing gradient problem (to some extent), as compared to sigmoid activation. The gradient of the tanh function is generally higher than that of the sigmoid, enabling better weight updates during backpropagation.

Recurrent Neuron

In RNNs the hidden state Hi is calculated for every input Xi to retain sequential dependencies. The computations follow these core formulas:



CNN Model Architecture

Key Differences Between CNN and RNN

- CNN is applicable for sparse data like images. An RNN is applicable to time-series and sequential data.
- While training the model, CNN uses a simple backpropagation, and RNN uses backpropagation through time to calculate the loss.
- RNN can have no restriction on the length of inputs and outputs, but CNN has finite inputs and outputs.
- CNN has a feedforward network, while RNN works on loops.
- CNN can also be used for video and image processing. RNN is primarily used for speech and text analysis.

Long Short-Term Memory (LSTM)

"Long-Term" memory (cell state) that persists over time with gating mechanisms (forget, input, and output gates).

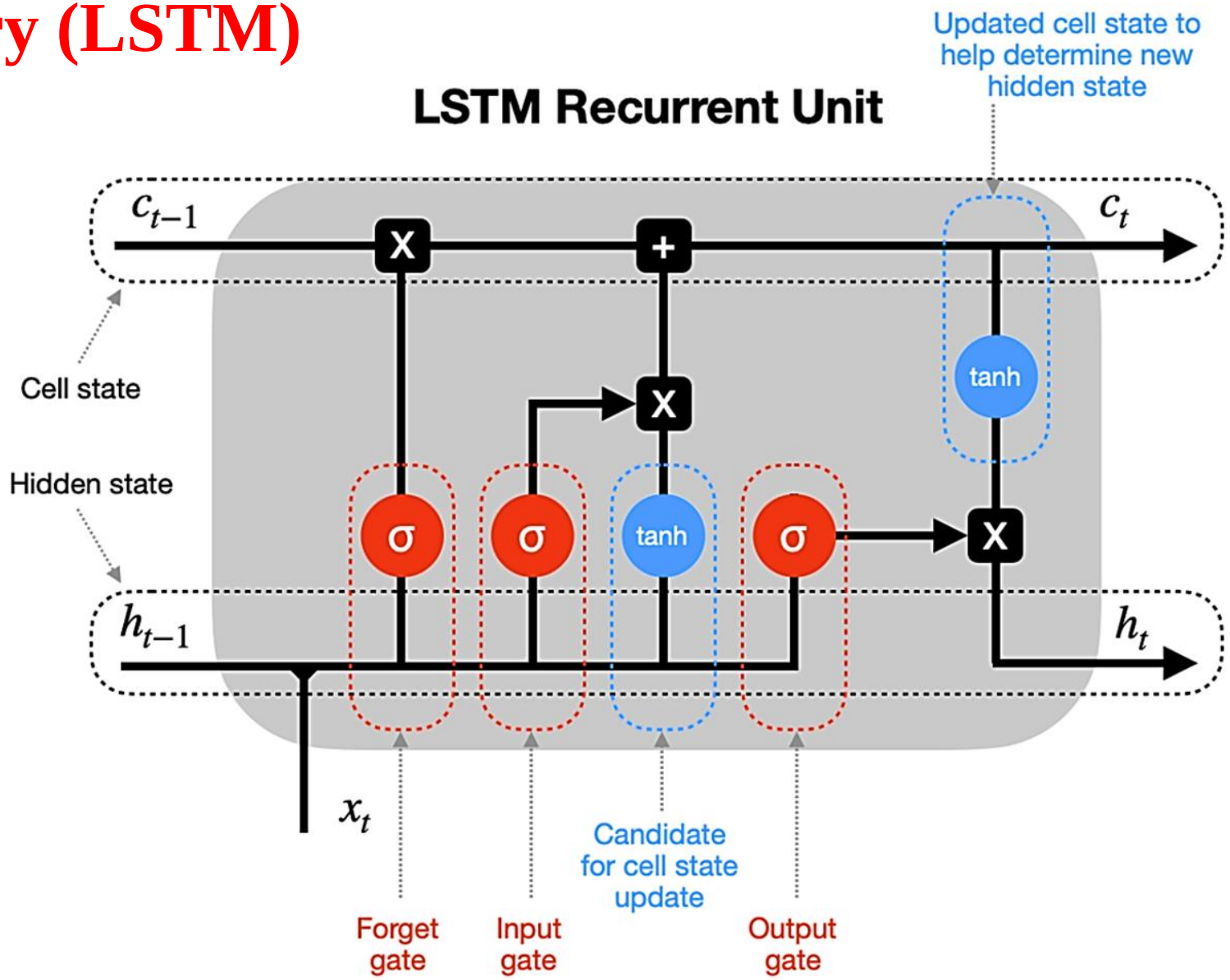
"Short-Term" memory (hidden state) to process data.

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c),$$

$$\mathbf{I}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i),$$
$$\mathbf{F}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f),$$
$$\mathbf{O}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o),$$

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t$$

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t)$$



Why we name this type of architecture as LSTM ?

- **Long-Term Component (Cell State):** The cell state acts like a conveyor belt, carrying relevant information across many time steps, allowing it to remember details from much earlier in a sequence.

- **Short-Term Component (Hidden State/Gates):** Similar to regular RNNs, LSTMs use hidden states for short-term information. However, they use gating mechanisms (input, forget, and output gates) to selectively decide what to add or remove from memory at each step.

Components of Long Short Term Memory (LSTM)

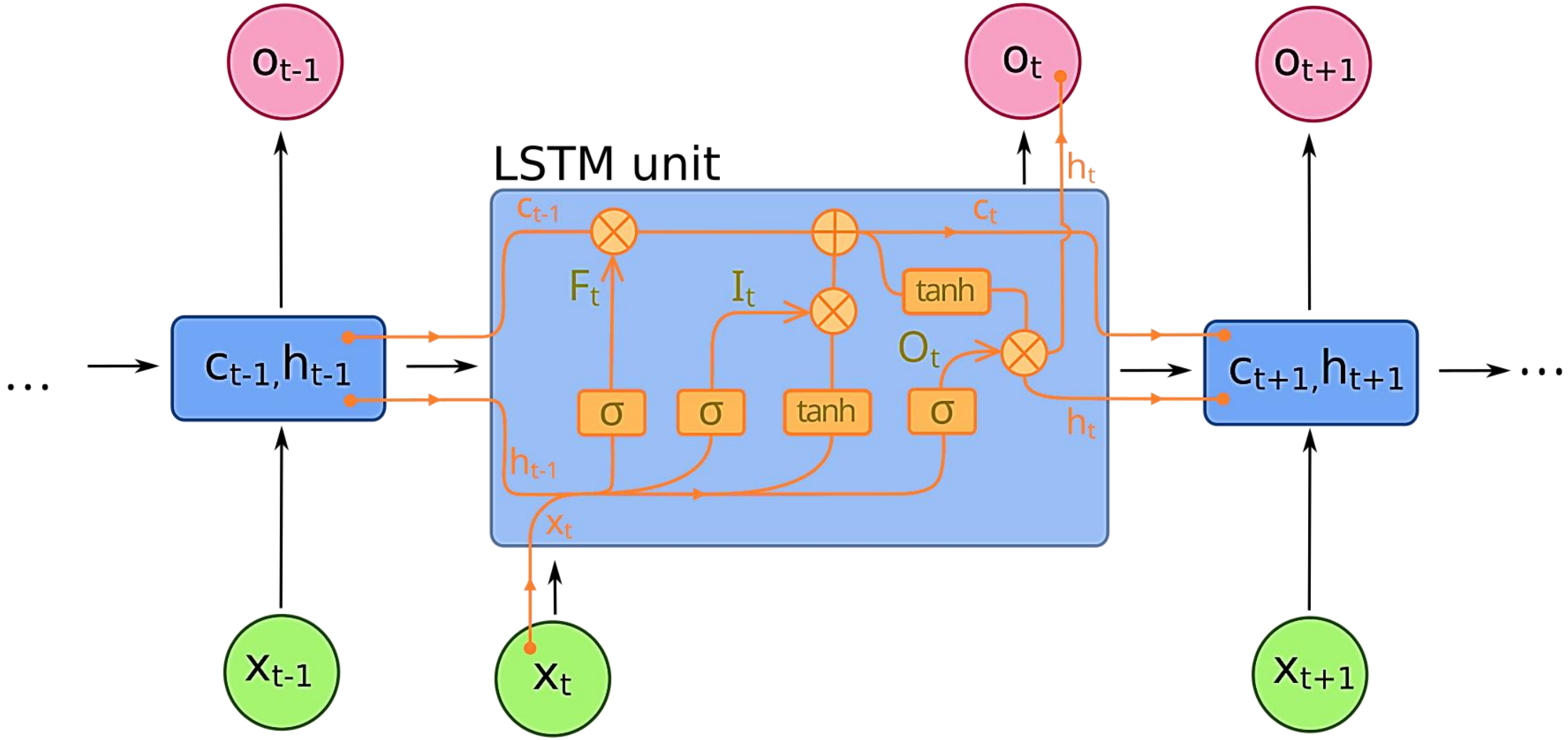
1. Three fully connected layers with sigmoid activation functions compute the value of input, forget, and output gates.
2. *Input Node*: typically computed with a *tanh* activation function to overcome the vanishing gradient issue.
3. *Input Gate:* determines how much of the input node's value should be added to the internal state of current memory cell.
4. *Forget Gate:* determines whether to keep the current value of the memory or flush it.
5. *Output Gate:* determines whether the memory cell should influence the output at the current time step.

Specific Features

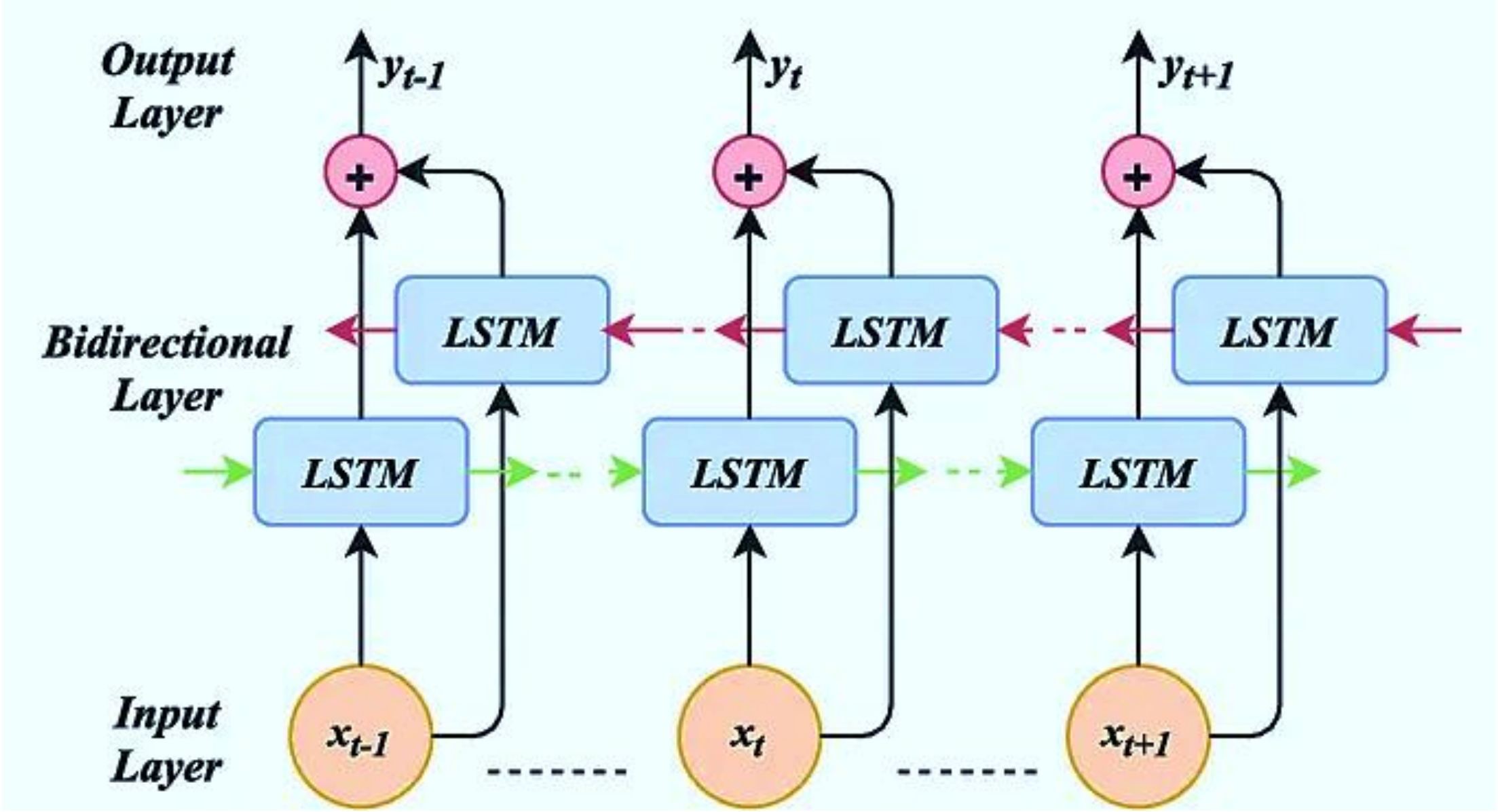
- **Solving Vanishing Gradient:** By maintaining a constant flow of information through special memory cells, LSTMs overcome the "vanishing gradient" problem where traditional RNNs forget long-term dependencies.
- **Analogy to Human Memory:** The name is an analogy to how humans use short-term memory to process immediate information while updating their long-term memory with key events.

Long Short Term Memory (LSTM)

They are especially effective for tasks where context and order matter.



Bidirectional Recurrent Neural Network (Bi-RNN)



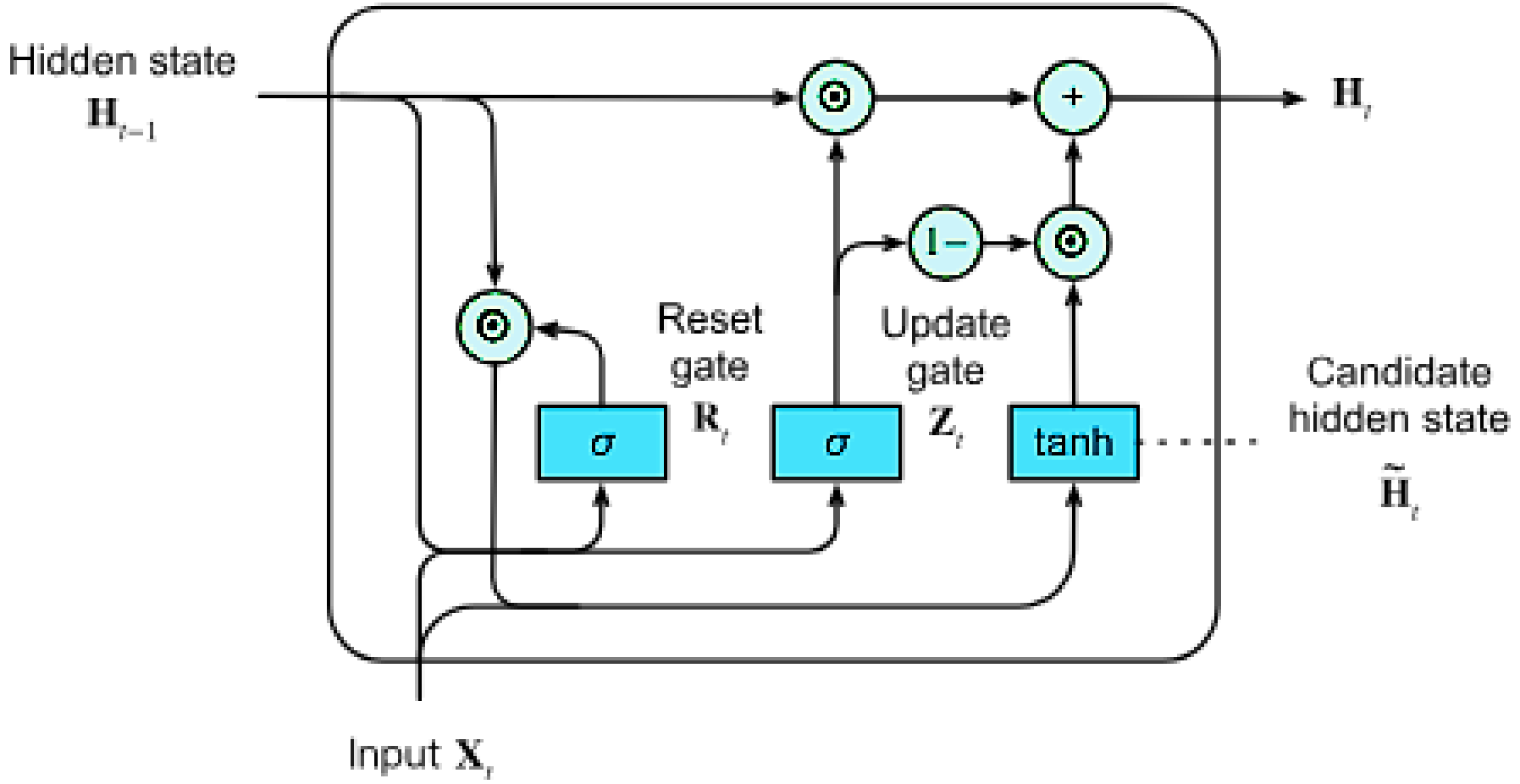
Forward Pass and Backward Pass

- **Forward Pass:** During the forward pass, the input sequence is fed into the forward LSTM layer from the first time step to the last. At each time step, the forward LSTM computes its hidden state and updates its memory cell based on the current input and the previous hidden state and memory cell.

- **Backward Pass:** Simultaneously, the input sequence is also fed into the backward LSTM layer in reverse order, from the last time step to the first. Similar to the forward pass, the backward LSTM computes its hidden state and updates its memory cell based on the current input and the previous hidden state and memory cell.

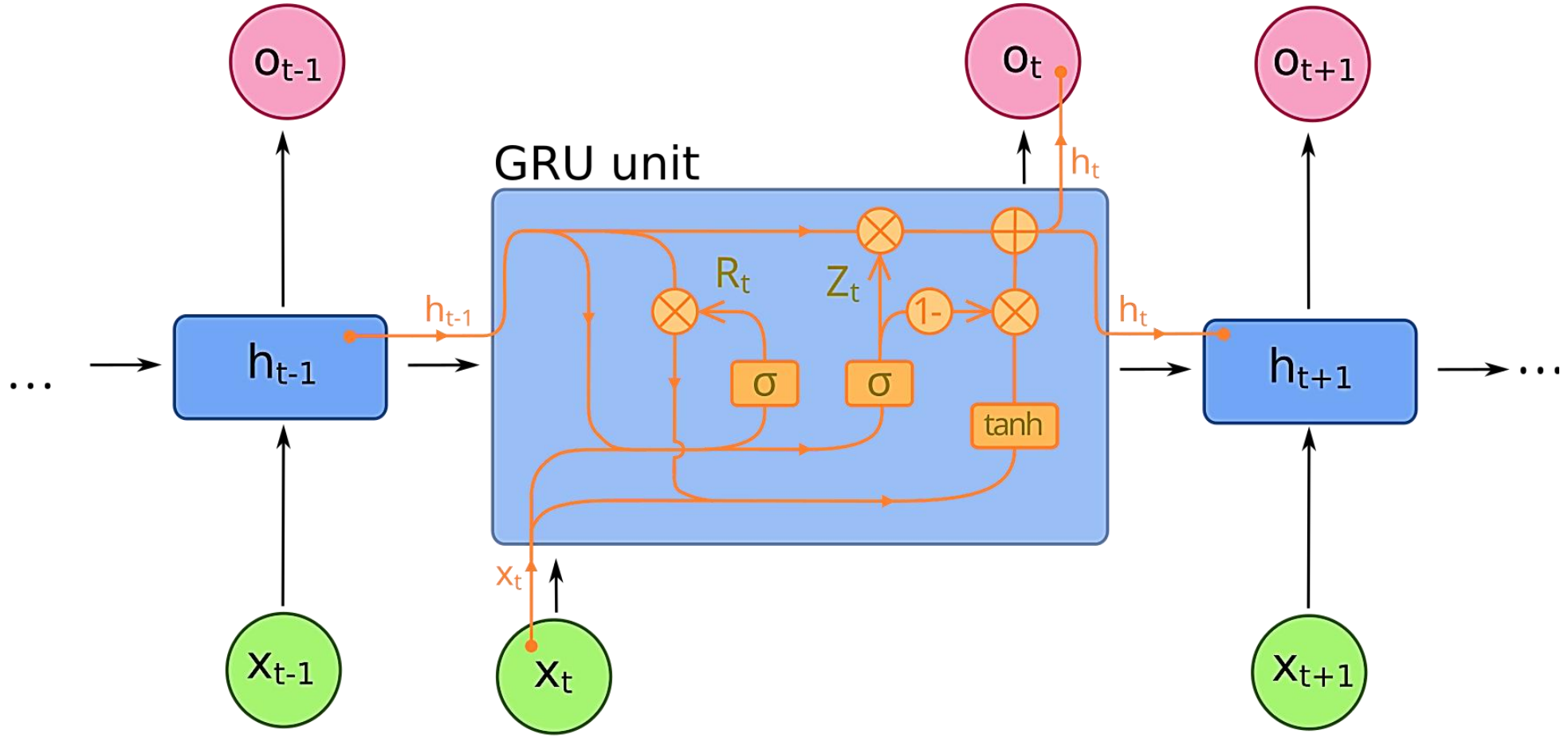
Gated Recurrent Unit (GRU)

This is a recurrent neural network (RNN) architecture similar to LSTM (Long Short-Term Memory).



Gated Recurrent Unit (GRU)

The candidate activation vector (often denoted as or) in a Gated Recurrent Unit (GRU) is a crucial component that represents the potential new hidden state at a given time step.



Important Points in GRU

- Like LSTM, GRU is designed to model sequential data by selectively remembering or forgetting information over time.
- GRU has a simpler architecture than LSTM, with fewer parameters, making it easier to train and more computationally efficient.
- The main difference between GRU and LSTM lies in how they handle the memory cell state.
- In LSTM, the memory cell state is maintained separately from the hidden state and is updated using three gates: the input, output, and forget gates.
- Whereas, in GRU, the memory cell state is replaced by a "candidate activation vector," which is updated using two gates: the reset and update gates.

https://en.wikipedia.org/wiki/Recurrent_neural_network

https://medium.com/@nivonl/exploring-visual-attention-in-transformer-models-ab538c06083a